CLAUDE CODE FULL COURSE 2026

Claude Code Full Course

Skills, MCPs, Subagents, and Everything That Actually Matters

Aniket Panjwani

2026

Part 1: Getting Started

1. **What Claude Code Actually Is**
   Demo: summarize a YouTube video, convert to PDF
2. **Pricing & Installation**
   Exercise: install tools + build a personal website in under 5 minutes
3. **The Terminal**
   Exercise: navigate folders, create and delete files
4. **The Interface**
   Exercise: run /usage, /context, /effort and read the output

Part 2: Essential Features

5. **Claude Code Guide**
   Demo: ask it a real question about Claude Code
6. **Voice Mode & Wispr Flow**
   Demo: voice-dictate "organize my Desktop into folders by type"
7. **Permissions**
   Demo: set permissions conversationally + cycle through four modes
8. **Hooks**
   Exercise: install destructive command guard
9. **CLAUDE.md**
   Why you probably shouldn't make one (and what to do instead)

Part 3: The Capability Toolbelt

10. **The Five Primitives:** CLI, Skills, MCPs, Plugins, Subagents
    Preview: connect to Gmail & Calendar with Google Workspace CLI
11. **Skills Deep Dive**
    Exercise: build a Google Workspace workflow skill (meeting prep or email digest)
12. **Skill vs MCP**
    Exercise: install and use a Notion MCP
13. **Plugins**
    Exercise: install the Compound Engineering plugin
14. **Subagents**
    Exercise: parallel Google Workspace workflow with subagents

Part 4: Building a Real App

15. **When to Use What:** Skills vs MCPs vs CLI vs Apps
16. **Git & GitHub**
    Exercise: git init, first commit, push to GitHub
17. **The Stack** — Next.js + Convex + Vercel
18. **Build the Job Tracker**
    Exercise: pipeline, cover letters, timeline — compound engineering process
19. **Build the CLI Companion Skill**
    Exercise: CLI + skill that talks to the same database
20. **Auth & Deploy**
    Exercise: Better Auth + ship it live on Vercel

SECTION 1

What Claude Code Actually Is

ChatGPT / Claude Web vs Claude Code

ChatGPT / Claude Web

Runs in a sandbox

SEALED SANDBOX

Run code snippets

Generate text

× Can't see your files

× Can't run commands

vs

Claude Code

Runs on your machine

YOUR ACTUAL COMPUTER

Read & edit your files

Run terminal commands

✓ Access your projects

✓ Install packages

SECTION 2

Pricing & Installation

Pricing & Installation

Subscription tiers (`claude.com/pricing`):

- Pro ($20/mo) — good to start, but you'll hit limits quickly
- Max 5x ($100/mo) — enough for most people
- Max 20x ($200/mo) — subagents, 1M context, higher limits

Install commands:

- Install Homebrew (`brew.sh`): `/bin/bash -c "$(curl -fsSL https://...)"`
- `npm install -g @anthropic-ai/claude-code` (Claude Code)
- `brew install yt-dlp pandoc` (useful CLI tools)
- `/plugin marketplace add anthropics/skills`
- `/plugin install example-skills@anthropic-agent-skills`

Windows users: install WSL first — see youtube.com/watch?v=Geq6RCYTSFA

SECTION 3

The Terminal

The Terminal

Three ways to use Claude Code:

1. **Standalone terminal** ← recommended
2. VS Code extension
3. Desktop app

Essential terminal commands:

- `cd` — change directory `ls` — list files `pwd` — print working directory
- `mkdir` — make folder `rm` — remove (careful!) `touch` — create file
- `clear` — clear screen `open .` — open current folder in Finder

SECTION 4

The Interface

The Interface

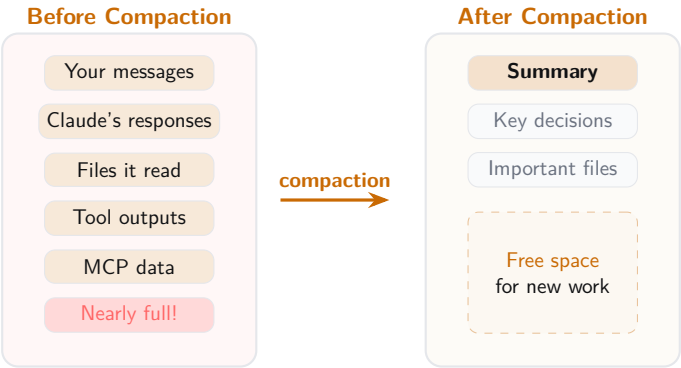
Slash commands to know:

- `/usage` — see how much of your plan you've used
- `/context` — see what's filling your context window
- `/effort` — set response quality (min / low / medium / high / max)
- `/help` — built-in help `/clear` — reset conversation
- `/compact` — summarize and compress the conversation so far

Configuration directories:

- `~/.claude/` — user-level settings (global)
- `.claude/` in a project — repo-level settings
- `settings.json` in either location for permissions and hooks

How Compaction Works



SECTION 5

The Claude Code Guide

The Claude Code Guide

- Claude Code includes a specialized "guide" subagent
- Ask it any question about Claude Code features, setup, or usage
- Examples: "How do I set up permissions?" or "What's the difference between skills and MCPs?"
- Uses web search to find up-to-date documentation
- Low cost — runs in its own context, doesn't clutter your main session

Tip: Use the guide throughout this course whenever you're confused about a feature.

SECTION 6

Voice Mode & Wispr Flow

Voice Mode & Wispr Flow

- Speaking is ~150 words/minute vs ~40 words/minute typing
- More natural for complex, multi-step instructions
- **Wispr Flow** — recommended dictation tool for macOS
- Works system-wide: dictate into any text field, including the terminal
- Claude Code processes voice-dictated prompts identically to typed ones

Example voice prompt:

"Look at all the files on my Desktop, sort them into folders by type — PDFs in one folder, images in another — and give me a summary of what you found."

SECTION 7

Permissions

Permissions & Permission Modes

Four permission modes (cycle with `Shift+Tab`):

1. **Default** — prompts on first use of each tool type
2. **Accept Edits** — auto-approves file edits, still asks for commands
3. **Plan Mode** — read-only, no file changes or commands
4. **YOLO / Bypass** — skips all prompts (opt-in only)

Enable YOLO mode:

Use `/update-config` or say "set my default permission mode to bypass permissions," then restart Claude Code. After that, `Shift+Tab` cycles through all four modes.

Fine-grained rules:

Run `/permissions` to see allow/ask/deny rules and which settings file they come from.

SECTION 8

Hooks

Hook Events

- **PreToolUse** — runs before Claude uses a tool
  - Block `rm -rf /` before it executes (Destructive Command Guard)
  - Prevent edits to `.env` or `package-lock.json`
- **PostToolUse** — runs after a tool succeeds
  - Auto-format your code with Prettier every time Claude edits a file
  - Run a linter after every code change
- **Stop** — runs when Claude finishes responding
  - Send yourself a desktop notification: "Claude is done!"
  - Useful when you walk away and let Claude work
- **SessionStart** — runs when you start a new session
  - Remind Claude of important project rules after compaction
  - "Always use Bun, not npm" — re-injected every session
- **UserPromptSubmit** — runs before Claude reads your message
  - Catch accidental pastes of API keys or passwords
  - Add context automatically before Claude sees your prompt

SECTION 9

CLAUDE.md

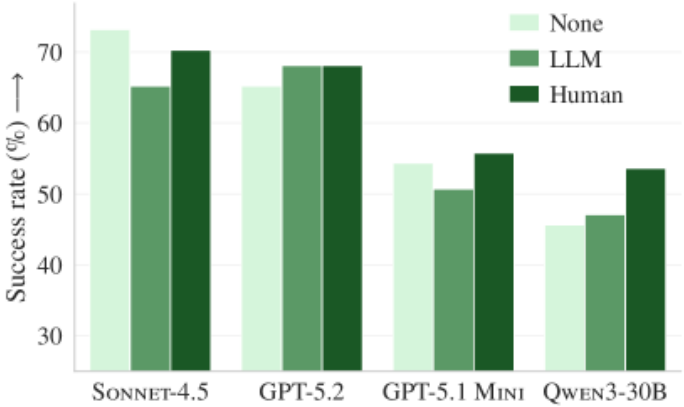
CLAUDE.md Levels

`some/path/CLAUDE.md`	That folder, on-demand
`./CLAUDE.md`	Your team, via git
`~/.claude/CLAUDE.md`	Just you, all projects

more specific

All levels load together — more specific wins on conflicts

Do AGENTS.md Files Actually Help?



Source: Gloaguen et al. (2026), "Evaluating AGENTS.md," ETH Zurich / INSAIT

The Cost of Context Files

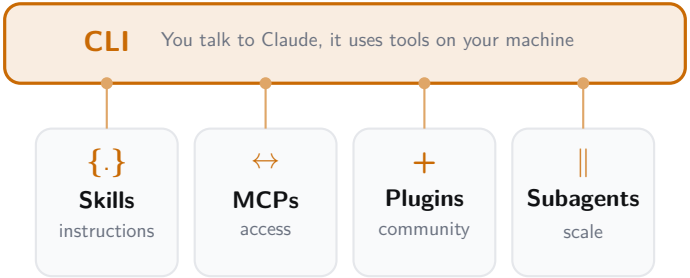
		Sonnet-4.5		GPT-5.2		GPT-5.1 M.		Qwen3-30B	
		Steps	Cost	Steps	Cost	Steps	Cost	Steps	Cost
SWE-Bench Lite	None	54.4	1.30	12.5	0.32	40.9	0.18	29.7	0.12
	LLM	57.2	1.51	12.7	0.43	45.2	0.22	32.2	0.13
Agent-bench	None	40.7	1.15	12.1	0.38	40.6	0.18	31.5	0.13
	LLM	46.5	1.33	13.1	0.57	46.9	0.20	34.2	0.15
	Hum.	45.3	1.30	13.6	0.54	46.6	0.19	32.8	0.15

LLM-generated files increase steps and cost across every model.

SECTION 10

The Five Practical Primitives

The Capability Toolbelt



SECTION 11

Skills Deep Dive

What Is a Skill?

Two definitions:

- **Intuitive:** Instructions we give Claude Code to help it learn to do specific things the way we want, every time
- **Technical:** A folder containing a markdown file with frontmatter, instructions, and optional reference files and scripts

Skill file structure:

- `skills/skill-name/skill-name.md` — main instructions (always loaded when invoked)
- `reference.md` — supplementary info (loaded only when needed)
- `scripts/` — helper scripts the skill can call

Key advantage: Skills only load when invoked — they don't consume context tokens until you need them.

Installing, Finding, and Creating Skills

Skill scopes:

- **Repo-level:** `.claude/skills/` — available only in this project
- **User-level:** `~/.claude/skills/` — available in all projects

Finding skills:

- Search GitHub: e.g., "Notion Skills Claude"
- Browse the Anthropic Example Skills plugin
- Use the **skill-creator** skill to build your own

Exercise: Build a Google Workspace email management skill using the skill-creator. The skill wraps around the Google Workspace CLI from section 10 — the CLI gives access, the skill gives workflow.

SECTION 12

Skill vs MCP

Skills vs MCPs

MCP (Model Context Protocol):

- A standard for AIs to communicate with external systems (Notion, Jira, Gmail, etc.)
- Loads a large JSON schema into context describing all available function calls
- More deterministic — tells Claude exactly what tools are available

Skill:

- Instructions for Claude Code on *how* to use tools and what workflows to follow
- Loads on demand — much lighter on context
- Easier to create and customize

General advice: just make a skill. Even when an MCP exists, you'll usually want a skill on top for your specific workflows.

SECTION 13

Plugins

Plugins & the Skills Marketplace

What a plugin is:

- Someone else's bundle of skills packaged for easy installation
- May also include MCPs, subagents, and hooks
- Install, inspect, and customize — don't treat plugins as black boxes

Key commands:

- `/plugin marketplace add <owner/repo>` — register a marketplace
- `/plugin install <plugin-name>` — install a plugin from a marketplace
- Browse installed plugins in `~/.claude/plugins/`

Recommended plugin: Compound Engineering
github.com/EveryInc/compound-engineering-plugin — workflow tools for ideation, planning, code review, and more.

SECTION 14

Subagents

Subagents

How subagents work:

- Each subagent runs in its own context window
- When Claude Code sees a task matching a subagent's description, it delegates automatically
- Subagents work independently and return results to the parent session

When to use subagents:

- Tasks that can be parallelized (e.g., analyze 14 YouTube channels simultaneously)
- Tasks that benefit from specialized context (e.g., code review, research)
- *Not* as a starting point — build the workflow as a skill first, then parallelize

You've already met one: The Claude Code Guide (section 5) is a built-in subagent. The Compound Engineering plugin uses subagents extensively for parallel code review and research.

SECTION 15

When to Use What: Skills vs MCPs vs CLI vs Apps

Common Confusions — Now That You've Used Them All

- **Skills vs CLAUDE.md**
  - CLAUDE.md = constraints and conventions ("use Bun, not npm")
  - Skills = workflows and procedures ("how to prep for a meeting")
- **Skills vs MCPs**
  - Skill = instructions for *how* to do a job
  - MCP = access to a system Claude can't reach otherwise
- **MCPs vs Plugins**
  - MCP = connection to one external system (Gmail, Notion)
  - Plugin = bundle of skills someone packaged for you to install
- **Skills vs Plugins**
  - Plugin = someone else's skills. You can also make your own.
  - Install a plugin as a starting point, customize from there.

SECTION 16

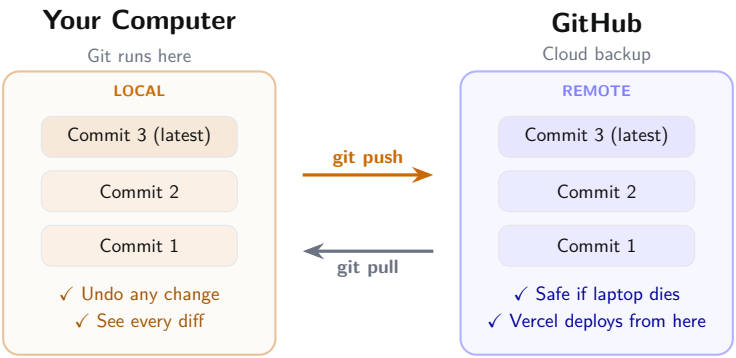
Git & GitHub

Git Maps to What You Already Do

What You Already Do	Git Equivalent
Create a project folder	`git init`
Save your work	`git commit`
Copy files to try something	`git branch`
Back up to Dropbox	`git push`

You only need the first two and the last one for this course.

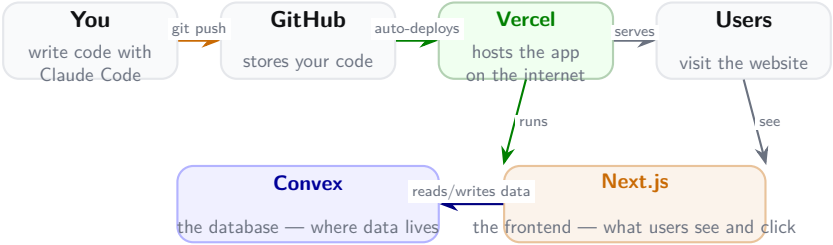
Git vs GitHub



SECTION 17

The Stack: Next.js + Convex + Vercel

Every Web App Has Three Layers



SECTION 18

Build the Job Application Tracker

Building the Job Tracker

Compound Engineering process:

1. `/ce:ideate` — generate and evaluate improvement ideas
2. `/ce:brainstorm` — explore requirements through dialogue
3. `/ce:plan` — structured implementation planning
4. `/ce:work` — execute the plan with quality checks

Features built:

- Kanban board with stages: Applied, Phone Screen, Interview, Offer, Rejected
- Drag-and-drop between stages, company details, notes

Key commands:

`npx convex dev` — run dev server locally `npx convex dashboard` — view data in browser

SECTION 19

Build the CLI Companion Skill

The CLI Companion Skill

Why a CLI companion?

- Web app is great for browsing — but slow for quick updates
- The CLI skill talks directly to the same Convex database
- Say: "add an application for Stripe as a data scientist" from the terminal

The pattern:

- Web app for visual browsing and drag-and-drop
- Claude Code skill for power-user terminal actions
- Same data, two interfaces

Possible extensions:

Job board scraping, LinkedIn integration, email follow-up reminders via Google Workspace.

SECTION 20

Auth & Deploy

Auth & Deploy Checklist

Authentication:

- Add login/signup with Better Auth (or similar)
- Protect routes so only logged-in users see their data
- Multi-user support: share with a career coach or partner

Deploy to production:

1. Set up Convex production deployment (separate from dev)
2. Generate a production admin key in the Convex dashboard
3. Create `.env.local` with production credentials
4. Push to GitHub → connect Vercel → app deploys automatically
5. Set environment variables in Vercel dashboard

The journey: Skill → app → deployed. Start with skills, graduate to apps when usage proves it.

Course Resources

Public Deck Bonus

- Full course outline and script: linked in description
- Claude Code pricing: claude.com/pricing
- Google Workspace CLI: github.com/googleworkspace/cli
- Compound Engineering plugin: github.com/EveryInc/compound-engineering-plugin
- Anthropic Example Skills: linked in description
- Destructive Command Guard: github.com/Dicklesworthstone/destructive_command_guard
- Wispr Flow (voice dictation): wispr.com
- Convex (database): convex.dev
- Vercel (hosting): vercel.com